

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И
РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ
по лабораторным работам
по дисциплине

Языковые процессоры интеллектуальных систем
Вариант 18

Выполнил

Немкова Е.А. гр. 2217011

Проверил

Соколович М. Г.

Минск 2025

Лабораторная работа №1

Задание: Написать 3 примера фрагментов кода языка по варианту, все свойства должны быть использованы, встроенные типы, операции и функции должны быть использованы и показаны на примерах.

Требования к разрабатываемому языку:

1. Встроенные типы не менее трех
2. Возможность инициализация переменных всех типов при объявлении:
<тип>? <имя_переменной> = <выражение>
 - 2.1. Инициализирующее выражение может быть константным
3. Встроенные операции не менее 10 штук
4. Встроенные функции
 - 4.1. Встроенные функции ввода\вывода для работы со встроенными типами read() write()
5. Использование сложных выражений (составных и со скобками)
6. Блочный оператор
7. Управляющие структуры
 - 7.1. Условный оператор (if-then-else)
 - 7.2. Операторы цикла (while и until)
 - 7.3. Оператор цикла с итерациями (for)
8. Пользовательские подпрограммы
 - 8.1. Передача и возврат параметров
 - 8.2. Задание локальной и глобальной области видимости для имен переменных

Язык:

1. Язык для работы с реляционными данными
 - 1.1. Встроенные типы: table, row, column
 - 1.2. Встроенные функции задания структур, изменения данных, поиска

Свойства языка:

1. Объявление переменных
Не явное
2. Преобразование типов
Явное, например, a = (int) b
3. Оператор присваивания
Одноцелевой, например, a = b
4. Структуры, ограничивающие область видимости
Подпрограммы
5. Маркер блочного оператора
Не явный, например как в python

6. Условные операторы
Двух вариантный оператор if-then-else
7. Перегрузка подпрограмм
Отсутствует
8. Передача параметров в подпрограмму
По ссылке
9. Допустимое место объявления подпрограмм:
В начале программы

Целевой код:

1. LLVM IR, формат промежуточного кода для LLVM (<http://llvm.org>)

Полученный язык:

Разработанный язык программирования является императивным, статически типизированным (с элементами вывода типов) и объектно-ориентированным. Он специализирован для обработки реляционных данных (таблиц).

1. Система типов

Язык поддерживает следующие категории типов данных:

- **Базовые примитивные типы:**
 - int - целочисленный тип (соответствует i32 в LLVM).
 - float - число с плавающей точкой двойной точности (соответствует double в LLVM).
 - string - строковый тип (указатель i8*).
 - void - пустой тип (используется для процедур, не возвращающих значения).
- **Специализированные типы (Domain-Specific):**
 - table - дескриптор реляционной таблицы. Хранит схему (колонки) и данные (строки). Реализован через указатель на структуру в Runtime-библиотеке.
 - row - динамическая структура, представляющая одну строку таблицы. Позволяет обращаться к полям по именам колонок (разрешение имен происходит в Runtime).
 - column - метаданные колонки таблицы.
- **Пользовательские типы (ООП):**

- struct - именованные структуры данных, поддерживающие инкапсуляцию полей и методов.

2. Объявление и инициализация переменных

В языке реализовано **неявное объявление переменных**. Тип переменной выводится автоматически из типа присваиваемого выражения при первом использовании.

- Синтаксис: <имя_переменной> = <выражение>
- Пример: x = 10 (тип x становится int).

3. Преобразование типов

Поддерживается **только явное** преобразование типов. Неявные преобразования (например, сложение числа и строки) запрещены семантическим анализатором.

- Синтаксис: (<целевой_тип>) <выражение>
- Пример: val = (float) 10

4. Структура программы и области видимости

- Программа состоит из объявлений структур (struct), глобальных подпрограмм (sub) и основного исполняемого кода.
- **Блочная структура:** Используется синтаксис, ориентированный на переносы строк (подобно Python), но с явными закрывающими маркерами (end if, end for, end sub) для устранения неоднозначности парсинга.
- **Области видимости:** Поддерживается иерархия: Глобальная область -> Область функции/метода -> Область цикла.

5. Объектно-ориентированные возможности

- **Инкапсуляция:** Объединение данных и методов внутри struct.
- **Наследование:** Поддерживается одиночное наследование через ключевое слово extends.
- **Полиморфизм:** Возможность переопределения методов в дочерних структурах.
- **Контекст:** Доступ к полям текущего объекта внутри методов осуществляется через ключевое слово self.

6. Управляющие конструкции

- **Условный оператор:** if <условие> : ... else : ... end if.
- **Цикл:** Итератор по строкам таблицы for row <имя> in <таблица> : ... end for.

Таблица приоритетов операторов:

Уровень	Операторы
1.	Первичные выражения ()
	Доступ к членам " . "
	Вызов функции ()
	Создание объекта new
2.	Приведение типов (type)
3.	Мультипликативные *, /
4.	Аддитивные +, -
5.	Сравнения >, <, >=, <=, ==, !=
6.	Присваивание =

Примечание к таблице:

- Оператор присваивания = в данной грамматике является частью инструкции (statement), а не выражения (expression), поэтому технически он не участвует в вычислении сложных выражений, но логически выполняется последним.
- Логические операторы (AND, OR) не были включены в базовую версию грамматики для упрощения, но при необходимости они заняли бы место между операторами сравнения и присваивания.

Дополнительные требования (усложнение):

- Поддержка объектно-ориентированного программирования (ООП).
- Пользовательские структуры данных (struct) с методами.
- Наследование и полиморфизм.

Фрагмент 1. Базовые операции с таблицами и подпрограммы Демонстрация работы со встроенными типами table, row и функциями манипуляции данными.

```
sub initialize_database(ref table db)
// Неявное объявление переменных
db = create_table("Employees")

// Работа со встроенными функциями
add_column(db, "id", "int")
add_column(db, "name", "string")
add_column(db, "salary", "float")

// Использование литералов строк (RowLiteral)
insert_row(db, {1, "Иван Иванов", 50000.0})
insert_row(db, {2, "Петр Петров", 45000.0})
end sub

// Основной код
company_db = create_table("Empty")
initialize_database(company_db)
print_table(company_db)
```

Фрагмент 2. ООП: Структуры, методы и наследование Демонстрация создания пользовательских типов данных, наследования и переопределения методов.

```
struct DataProcessor is
```

```

tableName : string

method setTableName(string name)
    self.tableName = name
end method

method process(ref table t)
    write("Base processing...")
end method
end struct

struct SalaryProcessor extends DataProcessor is
    minSalary : float

    // Полиморфизм (переопределение метода)
    method process(ref table t)
        write("Processing salaries for table: ", self.tableName)
        // Логика фильтрации...
    end method
end struct

// Использование
processor = new SalaryProcessor()
processor.setTableName("Employees")
processor.minSalary = 50000.0
processor.process(company_db)

```

Фрагмент 3. Управляющие конструкции и математика

Демонстрация циклов for, условий if-else и явного приведения типов.

```

sub calculate_stats(ref table t)
    total = 0.0
    count = 0

    // Цикл по строкам таблицы

```

```

for row r in t :
    // Явное преобразование типов (согласно варианту)
    val = (float) r.salary

    if val > 1000.0 :
        total = total + val
        count = count + 1
    else :
        write("Skipping low salary")
    end if
end for

if count > 0 :
    avg = total / (float) count
    write("Average: ", avg)
end if
end sub

```

Лабораторная работа №2

Задание: Составить файл грамматики с использованием ANTLR, можно отлаживать здесь(<http://labantlr.org/>), проверить на примерах из задания 1.1. Показывать выполнение задания в среде разработки.

Для реализации лексического и синтаксического анализатора использовался инструмент **ANTLR4**. Грамматика была адаптирована для поддержки Python-подобного синтаксиса (значимые переносы строк NL), но с использованием ключевых слов **end** для упрощения парсинга вложенных блоков.

Файл полученной грамматики:

```
grammar RelationalLang;
```


// --- ПАРСЕР ---

program

: (structDecl | subroutineDecl | statement | NL)* EOF
;

// --- Структуры ---

// struct Name is \n ... end struct \n

structDecl

: 'struct' ID ('extends' ID)? 'is' NL
(memberDecl | methodDecl | NL)*
'end' 'struct' NL
;

memberDecl

: ID ':' typeDecl NL
;

methodDecl

: 'method' ID '(' paramList? ')' NL
(statement | NL)*
'end' 'method' NL
;

// --- Подпрограммы ---

subroutineDecl

: 'sub' ID '(' paramList? ')' NL
(statement | NL)*
'end' 'sub' NL
;

paramList

: param (',' param)*
;

param

: 'ref'? typeDecl ID
;

statement

: assignStmt NL
| funcCallStmt NL
| methodCallStmt NL
| ifStmt NL
| forStmt NL
| returnStmt NL
;

assignStmt

: pathExpr '=' expr
;

pathExpr

: ID ('.' ID)*
;

funcCallStmt

: ID '(' exprList? ')'
;

methodCallStmt

: expr '.' ID '(' exprList? ')'
;

ifStmt

: 'if' expr ':' NL
 (statement | NL)*
 ('else' ':' NL (statement | NL)*)?
 'end' 'if'
;

forStmt

: 'for' typeDecl ID 'in' expr ':' NL

```

(statement | NL)*
'end' 'for'
;

returnStmt
: 'return' expr?
;

// --- Выражения ---
expr
: '(' typeDecl ')' expr      # CastExpr
| pathExpr                  # IdExpr
| literal                   # LiteralExpr
| '{' exprList? '}'         # RowLiteral
| ID '(' exprList? ')'      # FuncCallExpr
| expr '.' ID '(' exprList? ')' # MethodCallExpr
| 'new' ID '(' ' ' ')'      # NewExpr
| expr ('*' | '/') expr     # MulDivExpr
| expr ('+' | '-') expr     # AddSubExpr
| expr ('==' | '!=' | '>' | '<' | '>=' | '<=') expr # CompareExpr
| '(' expr ')'              # ParenExpr
;

exprList
: expr (',' expr)*
;

typeDecl
: 'int' | 'float' | 'string' | 'table' | 'row' | 'column' | 'void' | ID
;

literal
: INT | FLOAT | STRING
;

// --- ЛЕКЦИЯ ---

```

STRUCT: 'struct';
EXTENDS: 'extends';
IS: 'is';
METHOD: 'method';
END: 'end';
SUB: 'sub';
REF: 'ref';
IF: 'if';
ELSE: 'else';
FOR: 'for';
IN: 'in';
RETURN: 'return';
NEW: 'new';

TYPE_INT: 'int';
TYPE_FLOAT: 'float';
TYPE_STRING: 'string';
TYPE_TABLE: 'table';
TYPE_ROW: 'row';
TYPE_COLUMN: 'column';
TYPE_VOID: 'void';

ID: [a-zA-Z_][a-zA-Z0-9_]*;
INT: [0-9]+;
FLOAT: [0-9]+ '.' [0-9]+;
STRING: "" .*? "";

COMMENT: '/' ~[\r\n]* -> skip;

NL: [\r\n]+;

WS: [\t]+ -> skip;

LPAREN: '('; RPAREN: ')';
LBRACE: '{'; RBRACE: '}';
COMMA: ','; DOT: '.';
COLON: ':';

ASSIGN: '=';
PLUS: '+'; MINUS: '-'; STAR: '*'; SLASH: '/';
EQ: '=='; NEQ: '!='; GT: '>'; LT: '<'; GE: '>='; LE: '<=';

Лабораторная работа №3

Задание: Написать синтаксический анализатор, который выдает ошибки, тестируется на примерах из задания 1.1, протестировать на новых примерах, показывающих обработку ошибок.

Был разработан класс MyErrorListener, перехватывающий стандартные ошибки ANTLR и выводящий их в читаемом формате.

Тестирование обработки синтаксических ошибок:

Для тестирования использовались файлы с намеренно внесенными ошибками (пропущенные двоеточия, неверные ключевые слова, нарушение вложенности).

Пример файла с ошибками (test_errors.txt):

```
struct BrokenStruct is
    // Ошибка: забыли end method
    method bad()
        write("Error")
end struct

sub main()
    // Ошибка: пропущено двоеточие
    if x > 10
        write("X")
    end if
end sub
```

Результат работы синтаксического анализатора:

```
(.venv) PS C:\Users\Katerina Nemkova\PycharmProjects\YAPIS\YAPIS3.1> python main.py tests\errors\test_errors.txt
--- Анализ файла: tests\errors\test_errors.txt ---
НАЙДЕНЫ СИНТАКСИЧЕСКИЕ ОШИБКИ:
Ошибка синтаксиса [Строка 7:4]: mismatched input 'struct' expecting 'method'
Ошибка синтаксиса [Строка 11:13]: mismatched input '\r\n\r\n' expecting '.'
Ошибка синтаксиса [Строка 14:14]: missing ':' at '\r\n'
>>> Компиляция ПРЕРВАНА (Синтаксис).
```

Полный список проверяемых синтаксических ошибок:

1. Синтаксические ошибки (Syntactic Errors)

Эти ошибки выявляются на этапе парсинга (ANTLR). Они указывают на то, что код не соответствует грамматическим правилам языка.

1.1. Ошибки структуры программы

- **Нарушение вложенности блоков:** Попытка объявить метод (method) вне структуры или структуру внутри другой структуры.
- **Незавершенные блоки:** Отсутствие закрывающих ключевых слов end if, end for, end sub, end struct, end method.
- **Неверное завершение блока:** Например, использование end struct для закрытия блока if.
- **Неожиданный конец файла (<EOF>):** Если программа обрывается посередине конструкции или отсутствует обязательный перенос строки в конце файла.

1.2. Ошибки ключевых слов и идентификаторов

- **Использование несуществующих ключевых слов:** Например, написание function вместо sub или class вместо struct.
- **Опечатки в ключевых словах:** Например, retrun вместо return.
- **Отсутствие обязательных ключевых слов:** Пропуск in в цикле for или is в объявлении структуры.

1.3. Ошибки пунктуации

- **Пропущенное двоеточие (:):** В инструкциях if, else, for перед началом блока (например, if x > 0 вместо if x > 0 :).
- **Пропущенные разделители:** Отсутствие запятых между аргументами функции или параметрами.
- **Непарные скобки:** Открытая скобка (без закрывающей).

1.4. Ошибки выражений

- **Некорректные операторы:** Использование операторов, не предусмотренных грамматикой (например, ++ или ==, если они не были описаны).
- **Ошибки в левой части присваивания (L-value):** Попытка присвоить значение литералу (например, 5 = x).

Лабораторная работа №4

Задание: Написать семантический анализатор, который выдает ошибки, тестируется на примерах из задания 1.1, протестировать на новых примерах, показывающих обработку ошибок.

Реализованные механизмы:

1. **Таблица символов (SymbolTable):** Поддержка областей видимости (Global -> Function -> Block). Хранение информации о типах переменных, сигнатурах функций и структурах структур.
2. **Проверка типов:**
 - Запрет неявного приведения (например, int + string запрещено).
 - Проверка соответствия типов при присваивании.
 - Проверка типов аргументов функций.
3. **Проверка объявлений:**
 - Запрет использования необъявленных переменных.
 - Запрет повторного объявления переменных/структур.
 - Проверка наличия полей у структур.
4. **Обработка встроенных типов:** Особая обработка типа row, позволяющая динамический доступ к полям (так как структура БД может меняться).

Полный список проверяемых семантических ошибок:

Эти ошибки выявляются на этапе обхода дерева (Visitor). Они указывают на то, что код написан грамматически верно, но не имеет смысла или нарушает правила типизации.

2.1. Ошибки объявлений и областей видимости (Scoping)

- **Использование необъявленной переменной:** Попытка чтения или записи в переменную, которая не была инициализирована в текущей или глобальной области видимости.
- **Повторное объявление структуры:** Попытка создать две структуры с одинаковым именем.
- **Повторное объявление подпрограммы:** Попытка создать две функции/подпрограммы с одинаковым именем.
- **Неизвестный тип данных:** Использование несуществующей структуры при объявлении типа переменной или параметра (например, `x : UnknownStruct`).

2.2. Ошибки типизации (Type Checking)

- **Несовместимость типов при присваивании:** Попытка присвоить переменной значение другого типа (например, `int_var = "string"`).
- **Запрещенные неявные преобразования:** Попытка сложить число и строку (`int + string`) без явного приведения типов (согласно варианту задания).
- **Некорректные операции:** Попытка применить арифметические операции к неподходящим типам (например, вычитание строк).
- **Ошибки приведения типов (Casting):** Попытка привести несовместимые типы (логика реализована в генераторе кода, но семантический анализатор отслеживает базовые типы).

2.3. Ошибки работы со структурами (ООП)

- **Обращение к несуществующему полю:** Попытка доступа `obj.field`, если у структуры `obj` нет поля `field`.
- **Обращение к полю не-структуры:** Попытка использовать точечную нотацию для примитивных типов (например, `int_var.field`).
- **Наследование от несуществующей структуры:** Указание в `extends` имени структуры, которая не была объявлена.

2.4. Ошибки вызова функций и методов

- **Вызов несуществующей функции:** Обращение к функции, которая не была объявлена в глобальной области или среди встроенных функций.

- **Вызов переменной как функции:** Попытка добавить скобки () к переменной, которая не является функцией.
- **Несоответствие количества аргументов:** Вызов функции с меньшим или большим числом аргументов, чем указано в её сигнатуре (например, `add_column(db, "id")` вместо 3 аргументов).

2.5. Специфические ошибки предметной области

- **Ошибки встроенных типов:** Хотя для типа `gow` разрешен динамический доступ, попытка использовать его как структуру в контексте, где ожидается конкретный тип `struct`, будет отслежена.

Пример работы семантического анализатора

файл test_sem_types.txt

```
...
struct User is
    age : int
end struct

sub main()
    u = new User()

    // ОШИБКА: Присваивание строки в int
    u.age = "Too old"

    // ОШИБКА: Поле не существует
    u.salary = 1000
end sub
...
```

```
(.env) PS C:\Users\Katerina Nemkova\PycharmProjects\YAPIS\YAPIS3.1> python main.py tests\semantic_errors\test_sem_types.txt
--- Анализ файла: tests\semantic_errors\test_sem_types.txt ---
Синтаксис: ОК
НАЙДЕНЫ СЕМАНТИЧЕСКИЕ ОШИБКИ:
Семантическая ошибка [Строка 9]: Несовместимые типы: ожидался 'int', получен 'string'.
Семантическая ошибка [Строка 12]: У структуры 'User' нет поля 'salary'.
>>> Компиляция ПРЕРВАНА (Семантика).
-----
```

Лабораторная работа №5

Задание: Написать компилятор в исходный код по варианту и написать отчет.

Разработан модуль генерации кода CodeGenVisitor.py с использованием библиотеки llvmlite.

Архитектура компилятора:

1. **Frontend (Python):** Парсинг -> Семантический анализ -> Генерация LLVM IR (.ll файл).
2. **Runtime Library (C):** Файл runtime.c, реализующий сложные структуры данных (таблицы, динамические массивы) и функции ввода-вывода.
3. **Backend (Clang):** Компиляция .ll файла и линковка с runtime.c в исполняемый файл .exe.

Реализация генератора кода:

- Поддержка типов i32, double, i8* (строки), void* (таблицы).
- Генерация инструкций alloca, store, load для переменных.
- Генерация арифметических инструкций (add, fadd, mul и т.д.) с автоматическим выбором типа (целое/плавающее).
- Реализация управляющих конструкций (br, icmp, phi) для if-else.
- Вызов внешних функций (Foreign Function Interface) для работы с таблицами.

файл /test_db.txt

```
db = create_table("Employees")
add_column(db, "ID", "int")
add_column(db, "Name", "string")
add_column(db, "Salary", "float")

insert_row(db, {1, "Ivan", 50000.5})
insert_row(db, {2, "Petr", 45000.0})
insert_row(db, {3, "Maria", 60000.0})

write("Database created successfully.")
print_table(db)
```

```
(.venv) PS C:\Users\Katerina Nemkova\PycharmProjects\YAPIS\YAPIS3.1> python main.py tests/code/test_db.txt
--- Анализ файла: tests/code/test_db.txt ---
Синтаксис: ОК
Семантика: ОК. Код корректен.
Сгенерирован файл: tests/code/test_db.ll
Запуск компиляции: C:\msys64\clang64\bin\clang.exe tests/code/test_db.ll runtime.c -o tests/code/test_db.exe
warning: overriding the module target triple with x86_64-w64-windows-gnu [-Woverride-module]
1 warning generated.
УСПЕШНО СОЗДАН: tests/code/test_db.exe

--- ЗАПУСК ПРОГРАММЫ ---
[Runtime] Table 'Employees' created.
Database created successfully.

--- Table: Employees ---
ID      Name    Salary
1       Ivan    50000.50
2       Petr    45000.00
3       Maria   60000.00
```

Результаты работы компилятора.

Вывод:

- В ходе выполнения цикла лабораторных работ был разработан полнофункциональный компилятор для предметно-ориентированного языка работы с реляционными данными.
- Реализованы все этапы трансляции: лексический и синтаксический анализ (ANTLR4), семантический контроль (Visitor, Symbol Table) и генерация промежуточного кода LLVM IR.
- Особое внимание было уделено реализации объектно-ориентированных расширений (структуры, методы) и интеграции с библиотекой времени выполнения на языке C для поддержки сложных типов данных. Итоговый продукт генерирует нативные исполняемые файлы Windows.

Итоговый результат: Созданный компилятор успешно транслирует исходный код в исполняемые файлы формата .exe, которые корректно работают в среде Windows. Проект продемонстрировал работоспособность всех заложенных функций: от базовой арифметики и циклов до объектно-ориентированного взаимодействия и сложных операций над множествами.

В результате выполнения работ были получены практические навыки в области теории формальных языков, проектирования архитектуры компиляторов и

использования современных инструментов разработки языковых процессоров. Проект подтвердил, что выбранный стек технологий (Python + ANTLR4 + LLVM IR) является эффективным для быстрого прототипирования и реализации специализированных языков программирования.